

Frequency Mixer: Beauty and the Blur

EE200 Practical Assignment Q1 — Mukund Advait Balaji

July 5, 2025

Abstract

This practical assignment explores the creation of a frequency-mixer system. We aim to combine two images such that they are independently distinguishable based on proximity of viewer from the image, or based on whether the eye of the viewer is completely open or squinted.

1 Introduction

Our visual perception of images is seen to depend on the spatial frequencies of the image. High-frequency components of an image retain the fine details of the image. Whereas the low-frequency components of an image retain the overall shape and structure of the image.

This phenomenon can be seen by looking at an image from different distances (or by closing the eyelids by varying degrees). When you look at an image close up (or look at the image with wide open eyes), you can make out all the fine details of the image. However when we look at the image from afar (or look at the image with squinted eyes), you can only make out the rough details of the image.

We will use Python and a series of its libraries to simulate these effects, as well as create a hybrid image which uses these effects to show two images in one under different conditions.

2 2D Discrete Fourier Transform

Given an Image $I(x, y)$ is a finite aperiodic 2D discrete signal with intensity variations in both x and y directions. To analyze frequencies, we need to find 2D discrete variable Fourier transform of $I(x, y)$. The 2D Discrete Fourier Transform is given by:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x, y) \cdot e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

And the Inverse 2D Discrete Fourier Transform is given by:

$$I(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \cdot e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

We use the Fast Fourier Transform Algorithm for this purpose, while programming.

3 Magnitude Spectrum

The 2D Discrete Fourier Transform returns a complex double-parameter function $F(u, v)$, which represents the image in frequency domain. The magnitude of this function gives the spacial frequency data. The magnitude in normal form is given by:

$$|F(u, v)| = \sqrt{\text{Re}(F)^2 + \text{Im}(F)^2}$$

The magnitude in dB form is given by:

$$|F(u, v)|_{\text{dB}} = 20 \cdot \log_{10} (1 + |F(u, v)|)$$

4 Tools for Implementation

We will be using Python to manipulate the images, as well as the following libraries:

- NumPy: For Fourier transform, matrix and mathematical operations.
- Image (from PIL): For image manipulation, such as loading, resizing, cropping etc.
- Matplotlib: For graphical visualisation and image display.
- OpenCV: For image filtering.

5 Implementation

- Loading of Image into code: The images are loaded into the code using the Image library imported from PIL. They are converted to grayscale and resized for consistency.
- 2D Discrete Fourier Transform: Using the inbuilt function `fft2` from NumPy library, 2D Discrete Fourier Transform of images are calculated.
- Shifting of 2D Transform: Using the inbuilt function `fftshift` from NumPy library, lower frequency components are shifted to the center, which were otherwise present at the 4 corners.
- Magnitude Spectrums: The magnitude spectrum, in normal and dB form, of the transform is calculated using `abs` function of NumPy and plotted using matplotlib functions.

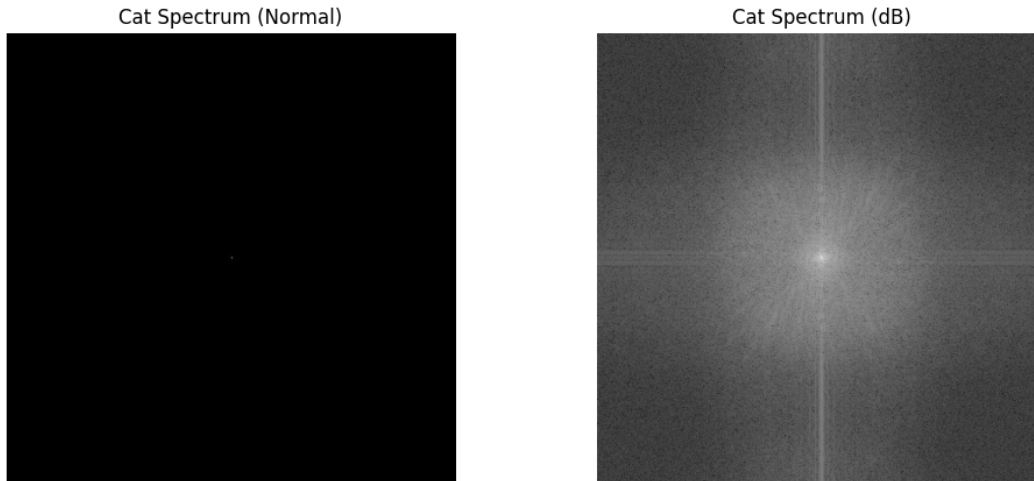


Figure 1: Magnitude Spectra of Cat Image (There is high intensity spot at the center of first plot)

- Image Rotation: Image is rotated by 90 degrees anti-clockwise and above analysis steps are again done for this image. Comparison is done between the magnitude spectra of the rotated and unrotated image. It is observed that rotating input image by 90 degrees counterclockwise also rotates the magnitude spectrum by 90 degrees in the same direction, but the overall frequency content remains the same.

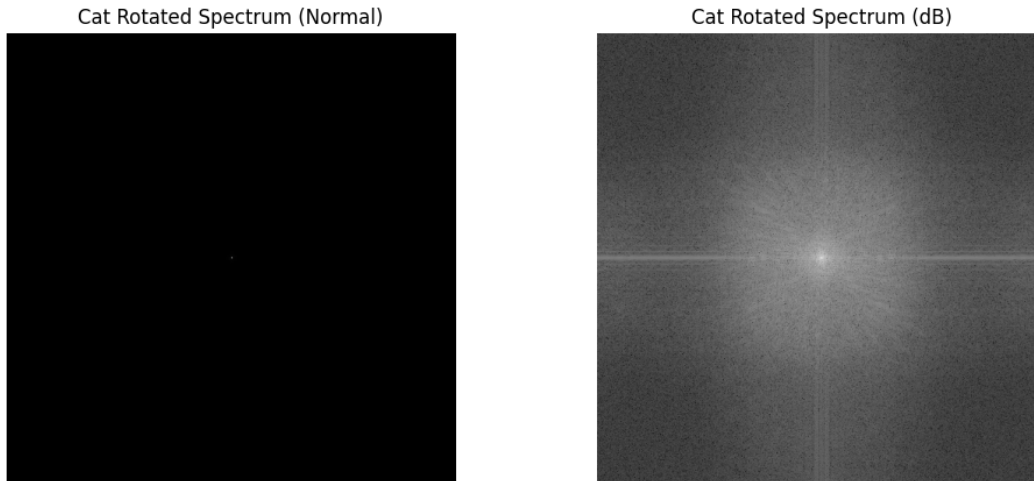


Figure 2: Magnitude Spectra of rotated Cat Image (There is high intensity spot at the center of first plot)

6 Filtering

1. Low Pass Filter: It is a type of filter which allows all signals below a certain frequency, but restricts signals above this frequency.

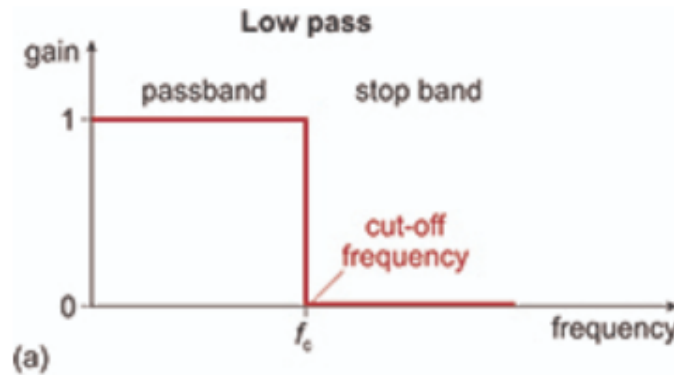


Figure 3: Low Pass Filter

In this assignment, it is implemented as a binary circular mask, with ones in the interior of the circle, and zeroes in the exterior, using OpenCV functions. When applied on the shifted magnitude spectrum, the low frequency components in the centre of the spectrum are caught within the circle and retained, while the high frequency components are removed.

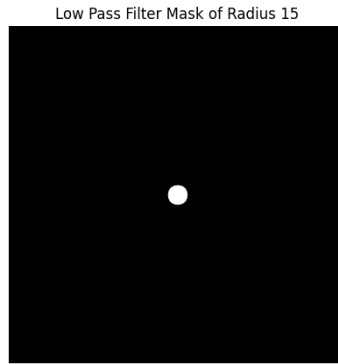


Figure 4: Low Pass Mask

2. High Pass Filter: It is a type of filter which allows all signals above a certain frequency, but restricts signals below this frequency.

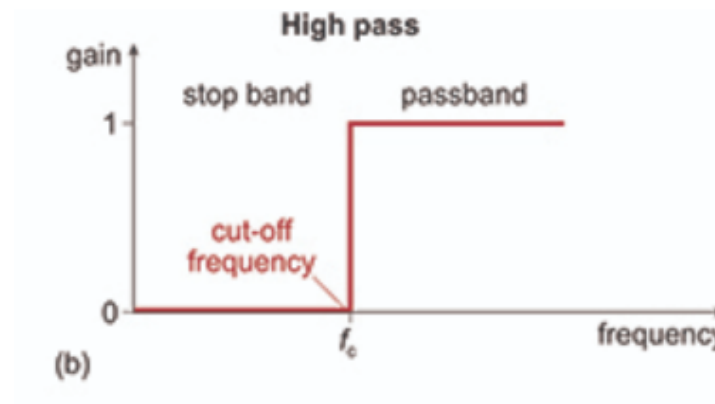


Figure 5: High Pass Filter

In this assignment, it is implemented as a binary circular mask, with zeroes in the interior of the circle, and ones in the exterior, using OpenCV functions. When applied on the shifted magnitude spectrum, the low frequency components in the centre of the spectrum are caught within the circle and removed, while the high frequency components are retained.

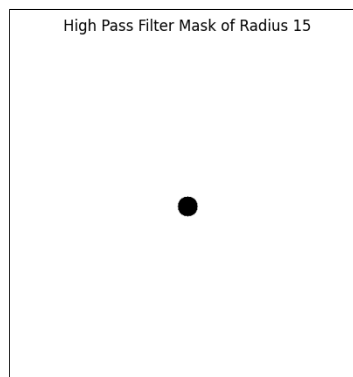


Figure 6: High Pass Mask

7 Hybrid Image Formation

Finally, the low pass filter is applied to one image (dog), while the high pass filter is applied to the other (cat). The images after filtering are combined to get the hybrid image.

$$HybridImage(x, y) = CatImage_{AfterHighPass}(x, y) + DogImage_{AfterLowPass}(x, y)$$

The results are normalised to the range of 0 to 255.



Figure 7: Hybrid Image

8 Conclusion

We can clearly see that in the final hybrid image, two images can be perceived based on viewing conditions. From far (or squinted eyes), the Dog is seen, due to low frequency content whereas from nearby (or wide open eyes), the Cat is seen, due to high frequency content. The radius of the mask affects how much of each image is seen. A radius of 1-2 gives an almost clear image of the cat whereas a radius of 30 gives an almost clear image of the dog. The midpoint, a mask of radius 15, seems to work well for our purpose.